반도체 품질관리 프로젝트
B 역할 — 2주차 코드 및 함수 상세 설명

목적

A가 결측치 기준으로 전처리한 Excel 데이터를 받아 정상(Pass)과 불량(Fail)의 변수별 분포를 탐색하고, Boxplot 및 요약 통계를 PDF로 정리하여 이상치 후보를 확인할 수 있도록 하는 작업이다.

※ 25%와 50%는 A가 준비한 두 데이터셋을 구분하는 기준이다. 이번 B 코드가 원본 데이터에 다시 결측률 기준을 적용해 열을 삭제하는 것은 아니다.

Ⅰ. 코드 1 — 결측치 25% 이상 제거 데이터

아래 코드는 Colab에서 하나의 코드 셀에 붙여넣어 실행하는 형태로 정리한 것이다. 실제 실행 코드에서는 25%와 50%의 입력 파일 및 결과 파일 이름만 구분한다.

```
# [■■ 1] ■■■ 25% ■■ ■■ ■■■
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files
from matplotlib.backends.backend_pdf import PdfPages

!apt-get -qq install fonts-nanum > /dev/null
import matplotlib.font_manager as fm
font_path = "/usr/share/fonts/truetype/nanum/NanumGothic.ttf"
fm.fontManager.addfont(font_path)
plt.rcParams["font.family"] = "NanumGothic"
plt.rcParams["axes.unicode_minus"] = False

print("■■■ 25% ■■ ■■ ■■■■ ■■■■■.")
uploaded = files.upload()
file_name = list(uploaded.keys())[0]
df = pd.read_excel(file_name)

feature_cols = [
    c for c in df.columns
    if str(c).startswith("Feature_")
    and pd.api.types.is_numeric_dtype(df[c])
]

def get_stats(x):
    x = x.dropna()
    if len(x) == 0:
        return dict(N=0, Mean=np.nan, Median=np.nan, Std=np.nan,
                    Min=np.nan, Max=np.nan, Outlier_N=0)
    q1, q3 = x.quantile(.25), x.quantile(.75)
    iqr = q3 - q1
    out = ((x < q1 - 1.5*iqr) | (x > q3 + 1.5*iqr)).sum()
    return dict(N=len(x), Mean=x.mean(), Median=x.median(),
                Std=x.std(), Min=x.min(), Max=x.max(),
                Outlier_N=int(out))

summary = []
for col in feature_cols:
    p = get_stats(df.loc[df["Result"]=="Pass", col])
    f = get_stats(df.loc[df["Result"]=="Fail", col])
    if pd.notna(p["Mean"]) and pd.notna(f["Mean"]):
        pooled = np.sqrt(np.nanmean([p["Std"]**2, f["Std"]**2]))
        diff = abs(p["Mean"]-f["Mean"])/pooled if pooled > 0 else 0
    else:
        diff = 0
    summary.append({
        "Feature": col,
        "Missing_Rate_%": df[col].isna().mean()*100,
        "Pass_N": p["N"], "Pass_Mean": p["Mean"],
        "Pass_Median": p["Median"], "Pass_Std": p["Std"],
        "Pass_Min": p["Min"], "Pass_Max": p["Max"],
        "Pass_Outlier_N": p["Outlier_N"],
        "Fail_N": f["N"], "Fail_Mean": f["Mean"],
        "Fail_Median": f["Median"], "Fail_Std": f["Std"],
        "Fail_Min": f["Min"], "Fail_Max": f["Max"],
        "Fail_Outlier_N": f["Outlier_N"],
        "Mean_Difference_Std": diff
    })

summary_df = pd.DataFrame(summary)
summary_df.to_csv(
    "B_2■■_25%_■■■_■■■■.csv",
    index=False, encoding="utf-8-sig"
)

pdf_name = "B_2■■_25%_■■■■_■■_■■■.pdf"
with PdfPages(pdf_name) as pdf:
    fig = plt.figure(figsize=(11.69, 8.27))
    ax = fig.add_axes([.12,.18,.76,.65])
    counts = df["Result"].value_counts()
    ax.bar(counts.index.astype(str), counts.values)
    ax.set_title("B 2■■ ■■ / ■■ ■■■ ■■", fontsize=18)
    ax.set_ylabel("■■ ■")
    for i,v in enumerate(counts.values):
        ax.text(i, v, f"{v:,}", ha="center", va="bottom")
    ax.grid(axis="y", alpha=.25)
    pdf.savefig(fig); plt.close(fig)

    top20 = summary_df.sort_values(
        "Mean_Difference_Std", ascending=False
    ).head(20).iloc[::-1]
    fig, ax = plt.subplots(figsize=(11.69,8.27))
    ax.barh(top20["Feature"], top20["Mean_Difference_Std"])
    ax.set_title("■■ / ■■ ■■ ■■■ ■ ■■ TOP 20", fontsize=16)
    ax.set_xlabel("|■■ ■■-■■ ■■| / ■■■■")
    ax.grid(axis="x", alpha=.25)
    fig.subplots_adjust(left=.16,right=.96,top=.92,bottom=.10)
    pdf.savefig(fig); plt.close(fig)

    for start in range(0, len(feature_cols), 12):
        cols = feature_cols[start:start+12]
        fig, axes = plt.subplots(4,3,figsize=(11.69,8.27))
        axes = axes.ravel()
        for ax,col in zip(axes,cols):
            normal = df.loc[df["Result"]=="Pass", col].dropna()
            fail = df.loc[df["Result"]=="Fail", col].dropna()
            data, labels = [], []
            if len(normal): data.append(normal.values); labels.append("Pass")
            if len(fail): data.append(fail.values); labels.append("Fail")
```

```
            if data: ax.boxplot(data, tick_labels=labels, showfliers=True)
            ax.set_title(
                f"{col}\n■■■: {df[col].isna().mean()*100:.1f}%",
                fontsize=8
            )
            ax.tick_params(axis="both", labelsize=7)
            ax.grid(axis="y", alpha=.18)
        for ax in axes[len(cols):]: ax.axis("off")
        fig.suptitle(
            f"■■■ ■■ / ■■ Boxplot "
            f"({start+1}~{min(start+12,len(feature_cols))}/{len(feature_cols)})",
            fontsize=12
        )
        fig.subplots_adjust(
            left=.06,right=.98,top=.91,bottom=.07,
            wspace=.35,hspace=.65
        )
        pdf.savefig(fig); plt.close(fig)

files.download(pdf_name)
files.download("B_2■■_25%_■■■_■■■■.csv")
```

Ⅱ. 코드 2 — 결측치 50% 이상 제거 데이터

아래 코드는 Colab에서 하나의 코드 셀에 붙여넣어 실행하는 형태로 정리한 것이다. 실제 실행 코드에서는 25%와 50%의 입력 파일 및 결과 파일 이름만 구분한다.

```
# [■■ 1] ■■■ 50% ■■ ■■ ■■■
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files
from matplotlib.backends.backend_pdf import PdfPages

!apt-get -qq install fonts-nanum > /dev/null
import matplotlib.font_manager as fm
font_path = "/usr/share/fonts/truetype/nanum/NanumGothic.ttf"
fm.fontManager.addfont(font_path)
plt.rcParams["font.family"] = "NanumGothic"
plt.rcParams["axes.unicode_minus"] = False

print("■■■ 50% ■■ ■■ ■■■■ ■■■■■.")
uploaded = files.upload()
file_name = list(uploaded.keys())[0]
df = pd.read_excel(file_name)

feature_cols = [
    c for c in df.columns
    if str(c).startswith("Feature_")
    and pd.api.types.is_numeric_dtype(df[c])
]

def get_stats(x):
    x = x.dropna()
    if len(x) == 0:
        return dict(N=0, Mean=np.nan, Median=np.nan, Std=np.nan,
                    Min=np.nan, Max=np.nan, Outlier_N=0)
    q1, q3 = x.quantile(.25), x.quantile(.75)
    iqr = q3 - q1
    out = ((x < q1 - 1.5*iqr) | (x > q3 + 1.5*iqr)).sum()
    return dict(N=len(x), Mean=x.mean(), Median=x.median(),
                Std=x.std(), Min=x.min(), Max=x.max(),
                Outlier_N=int(out))

summary = []
for col in feature_cols:
    p = get_stats(df.loc[df["Result"]=="Pass", col])
    f = get_stats(df.loc[df["Result"]=="Fail", col])
    if pd.notna(p["Mean"]) and pd.notna(f["Mean"]):
        pooled = np.sqrt(np.nanmean([p["Std"]**2, f["Std"]**2]))
        diff = abs(p["Mean"]-f["Mean"])/pooled if pooled > 0 else 0
    else:
        diff = 0
    summary.append({
        "Feature": col,
        "Missing_Rate_%": df[col].isna().mean()*100,
        "Pass_N": p["N"], "Pass_Mean": p["Mean"],
        "Pass_Median": p["Median"], "Pass_Std": p["Std"],
        "Pass_Min": p["Min"], "Pass_Max": p["Max"],
        "Pass_Outlier_N": p["Outlier_N"],
        "Fail_N": f["N"], "Fail_Mean": f["Mean"],
        "Fail_Median": f["Median"], "Fail_Std": f["Std"],
        "Fail_Min": f["Min"], "Fail_Max": f["Max"],
        "Fail_Outlier_N": f["Outlier_N"],
        "Mean_Difference_Std": diff
    })

summary_df = pd.DataFrame(summary)
summary_df.to_csv(
    "B_2■■_50%_■■■_■■■■.csv",
    index=False, encoding="utf-8-sig"
)

pdf_name = "B_2■■_50%_■■■■_■■_■■■.pdf"
with PdfPages(pdf_name) as pdf:
    fig = plt.figure(figsize=(11.69, 8.27))
    ax = fig.add_axes([.12,.18,.76,.65])
    counts = df["Result"].value_counts()
    ax.bar(counts.index.astype(str), counts.values)
    ax.set_title("B 2■■ ■■ / ■■ ■■■ ■■", fontsize=18)
    ax.set_ylabel("■■ ■")
    for i,v in enumerate(counts.values):
        ax.text(i, v, f"{v:,}", ha="center", va="bottom")
    ax.grid(axis="y", alpha=.25)
    pdf.savefig(fig); plt.close(fig)

    top20 = summary_df.sort_values(
        "Mean_Difference_Std", ascending=False
    ).head(20).iloc[::-1]
    fig, ax = plt.subplots(figsize=(11.69,8.27))
    ax.barh(top20["Feature"], top20["Mean_Difference_Std"])
    ax.set_title("■■ / ■■ ■■ ■■■ ■ ■■ TOP 20", fontsize=16)
    ax.set_xlabel("|■■ ■■-■■ ■■| / ■■■■")
    ax.grid(axis="x", alpha=.25)
    fig.subplots_adjust(left=.16,right=.96,top=.92,bottom=.10)
    pdf.savefig(fig); plt.close(fig)

    for start in range(0, len(feature_cols), 12):
        cols = feature_cols[start:start+12]
        fig, axes = plt.subplots(4,3,figsize=(11.69,8.27))
        axes = axes.ravel()
        for ax,col in zip(axes,cols):
            normal = df.loc[df["Result"]=="Pass", col].dropna()
            fail = df.loc[df["Result"]=="Fail", col].dropna()
            data, labels = [], []
            if len(normal): data.append(normal.values); labels.append("Pass")
            if len(fail): data.append(fail.values); labels.append("Fail")
```

```
            if data: ax.boxplot(data, tick_labels=labels, showfliers=True)
            ax.set_title(
                f"{col}\n■■■: {df[col].isna().mean()*100:.1f}%",
                fontsize=8
            )
            ax.tick_params(axis="both", labelsize=7)
            ax.grid(axis="y", alpha=.18)
        for ax in axes[len(cols):]: ax.axis("off")
        fig.suptitle(
            f"■■■ ■■ / ■■ Boxplot "
            f"({start+1}~{min(start+12,len(feature_cols))}/{len(feature_cols)})",
            fontsize=12
        )
        fig.subplots_adjust(
            left=.06,right=.98,top=.91,bottom=.07,
            wspace=.35,hspace=.65
        )
        pdf.savefig(fig); plt.close(fig)

files.download(pdf_name)
files.download("B_2■■_50%_■■■_■■■■.csv")
```

III. 코드 전체 흐름

단계	코드의 역할	결과
1	라이브러리 불러오기	pandas, numpy, matplotlib 등 준비
2	한글 폰트 설정	PDF 그래프의 한글 깨짐 방지
3	Excel 업로드	Colab에 파일 선택 및 업로드
4	Excel 읽기	DataFrame으로 데이터 저장
5	Feature 선택	Feature_로 시작하는 숫자형 변수만 선택
6	Pass / Fail 분리	정상과 불량의 분포를 별도로 계산
7	기초통계 계산	평균 · 중앙값 · 표준편차 · 최소 · 최대
8	IQR 이상치 후보 계산	1.5×IQR 기준 후보 개수 확인
9	평균 차이 계산	정상/불량 차이가 큰 변수 탐색
10	시각화	샘플 수, TOP 20, Feature별 Boxplot
11	PDF/CSV 저장	A에게 전달할 결과물 생성

IV. 주요 함수 · 명령어 설명

import pandas as pd — Excel 데이터를 표 형태(DataFrame)로 다루기 위한 pandas를 불러온다.

import numpy as np — NaN(결측값), 제곱근, 평균 등 수치 계산에 사용한다.

import matplotlib.pyplot as plt — 그래프를 만들고 제목 · 축 · 눈금 등을 설정한다.

files.upload() — Google Colab에서 사용자가 PC의 Excel 파일을 선택하도록 파일 선택창을 연다.

pd.read_excel(file_name) — 업로드한 Excel 파일을 pandas DataFrame으로 읽는다.

df['Result'].value_counts() — Result 열의 Pass와 Fail이 각각 몇 개인지 센다.

df[col].isna().mean() — 해당 Feature에서 결측값이 차지하는 비율을 계산한다.

dropna() — 통계 계산이나 Boxplot을 만들 때 해당 변수의 결측값을 제외한다.

quantile(0.25), quantile(0.75) — 1사분위수 Q1과 3사분위수 Q3를 구한다.

IQR = Q3 - Q1 — 사분위범위(IQR)를 계산한다.

1.5 × IQR — Boxplot에서 흔히 사용하는 통계적 이상치 판정 경계 계산에 사용한다.

ax.boxplot(...) — 정상/불량 데이터의 중앙값, 분포 범위, 극단값 후보를 시각화한다.

sort_values(...).head(20) — 정상/불량 평균 차이가 큰 Feature를 정렬하여 상위 20개를 선택한다.

PdfPages(pdf_name) — 여러 그래프를 하나의 PDF 파일에 페이지별로 저장한다.

pdf.savefig(fig) — 현재 그래프(fig)를 PDF의 한 페이지로 저장한다.

files.download(...) — Colab에서 생성된 PDF/CSV를 사용자의 컴퓨터로 다운로드한다.

Ⅴ. 핵심 계산식 이해하기

① 결측률

결측률 = 결측값 개수 ÷ 전체 데이터 개수 × 100. 현재 분석에서는 A가 전처리한 파일에 남아 있는 결측률도 그래프 제목에 표시한다.

② IQR 기반 이상치 후보

IQR = Q3 − Q1
하한 = Q1 − 1.5 × IQR
상한 = Q3 + 1.5 × IQR
하한보다 작거나 상한보다 큰 값은 통계적인 이상치 후보로 센다.

③ 정상/불량 평균 차이 지표

Mean Difference = |Pass 평균 − Fail 평균| ÷ 두 집단 표준편차의 결합값. 값이 클수록 정상과 불량의 중심 위치가 상대적으로 많이 떨어져 있다는 뜻이므로 시각화 우선순위를 정하는 데 사용한다.

Ⅵ. PDF 그래프를 읽는 방법

1페이지: Pass와 Fail의 전체 샘플 수를 확인한다.

2페이지: 정상/불량 평균 차이가 상대적으로 큰 Feature TOP 20을 확인한다.

이후 페이지: 각 Feature의 Pass/Fail Boxplot을 비교한다.

Boxplot의 상자 가운데 선은 중앙값, 상자는 Q1~Q3 범위, 수염 바깥의 점은 극단값 후보이다.

Pass와 Fail의 중앙값 위치가 다르거나 분포 폭이 크게 다르면 해당 변수에 정상/불량 차이가 있을 가능성을 검토한다.

Ⅶ. B 역할에서 반드시 주의할 점

- 통계적 이상치 후보를 실제 측정 오류나 공정 이상이라고 단정하면 안 된다.
- A는 원본 데이터, 장비 상태, 측정 조건, 공정 조건 등을 함께 확인한 뒤 최종 제거 여부를 판단한다.
- 이번 코드는 이상치를 자동 삭제하지 않는다. B는 A의 판단을 돕는 시각화 · 통계 자료를 제공한다.
- 25%와 50% 두 결과는 같은 분석 흐름으로 만들어 비교 가능하도록 구성한다.

Ⅷ. 최종 산출물

파일	용도
B_2주차_25%_정상불량_분포_시각화.pdf	25% 데이터의 정상/불량 분포 및 Boxplot
B_2주차_50%_정상불량_분포_시각화.pdf	50% 데이터의 정상/불량 분포 및 Boxplot
B_2주차_25%_변수별_통계요약.csv	25% 데이터의 Feature별 통계 및 이상치 후보 개수
B_2주차_50%_변수별_통계요약.csv	50% 데이터의 Feature별 통계 및 이상치 후보 개수